

CSC-364: Assignment 3 Report

1 Introduction

In this mini-report, I answer the congestion control mechanisms I implemented, timeout and retransmissions, provide insights on my generated graphs, and talk about some of the issues I encountered while working on this project.

2 Congestion Control

The client uses the three TCP Reno-style congestion control mechanisms that were specified in the Canvas assignment.

2.1 Slow Start

The congestion window, `cwnd`, starts at 1 packet. For each new ACK, `cwnd` increases by 1. This creates fast growth at the beginning of the transfer. Slow start continues until `cwnd` reaches `ssthresh`.

2.2 Congestion Avoidance

After `cwnd` reaches `ssthresh`, the client enters congestion avoidance. In this phase, `cwnd` increases by $1/\text{cwnd}$ for each new ACK. This makes the window grow more slowly, about 1 packet per RTT.

2.3 Fast Retransmit

The client tracks duplicate ACKs. If the same ACK is received three times, the client assumes that packet is missing. It retransmits the missing packet immediately instead of waiting for a timeout. After fast retransmit, `ssthresh` is set to half of `cwnd`, and `cwnd` is reduced to `ssthresh`.

3 Timeouts and Retransmissions

We are asked in the assignment that the client should use a 500ms socket timeout. If no ACK is received before the timeout, the client retransmits the oldest unacknowledged packet. On timeout, `ssthresh` is set to half of `cwnd`, and `cwnd` is reset to 1. Interestingly, I noted that some variations of Reno TCP use a "Fast

Recovery” mechanism, where $\text{cwnd} = \text{ssthresh} + 3$. I compared this mechanism to our Lecture 7, including the graphs, and noted that this additional feature is not utilized, as discussed in lecture.

The client stores unacknowledged packets in a dictionary in my implementation. This allows the client to resend packets after timeouts or after fast retransmit.

4 Server Behavior

The server uses cumulative ACKs. It tracks the next expected sequence number. If packets arrive out of order, the server buffers them. It only writes data to the output file when all previous bytes have been received. This allows the server to correctly rebuild the file even when packets arrive out of order.

The server also simulates packet loss using a configurable loss probability. The experiments used 1%, 10%, and 50% packet loss.

5 Graphs and Observations

5.1 Congestion Window

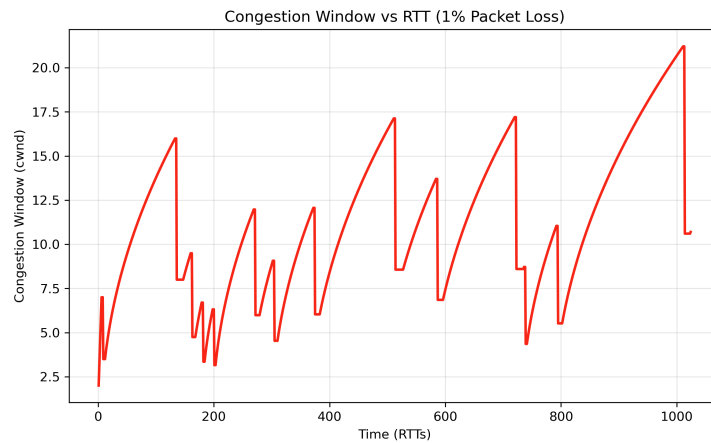


Figure 1: cwnd vs RTT with 1% packet loss

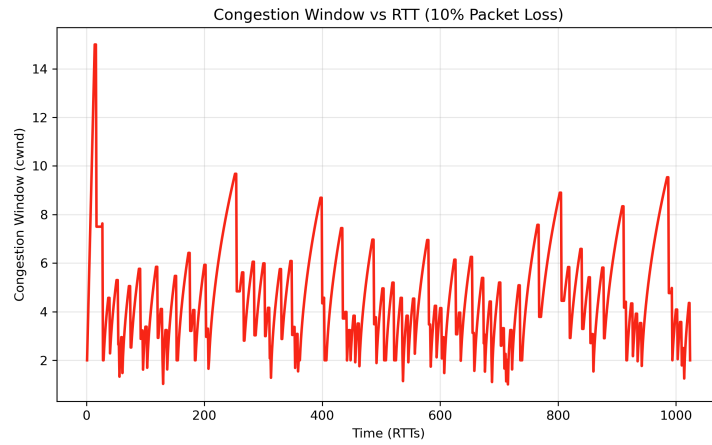


Figure 2: cwnd vs RTT with 10% packet loss

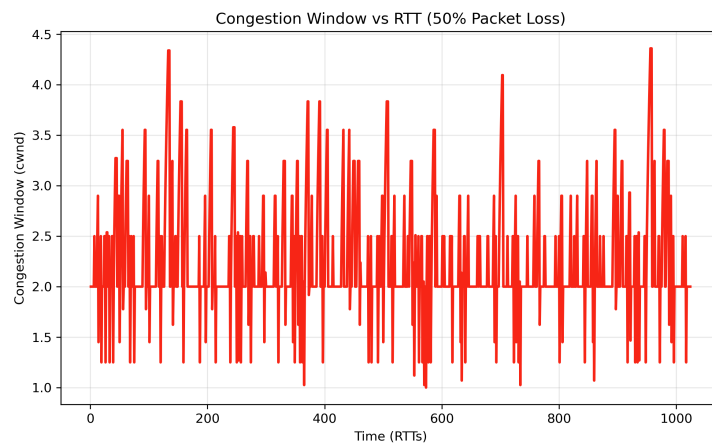


Figure 3: cwnd vs RTT with 50% packet loss

At 1% packet loss, `cwnd` grows higher because losses are less frequent. At 10% packet loss, `cwnd` drops more often and stays lower. At 50% packet loss, `cwnd` stays very small because the client repeatedly detects loss through duplicate ACKs and timeouts.

5.2 Retransmissions

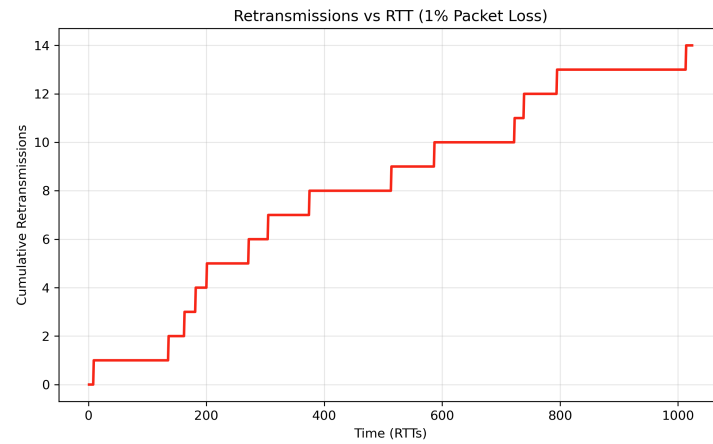


Figure 4: Retransmissions vs RTT with 1% packet loss

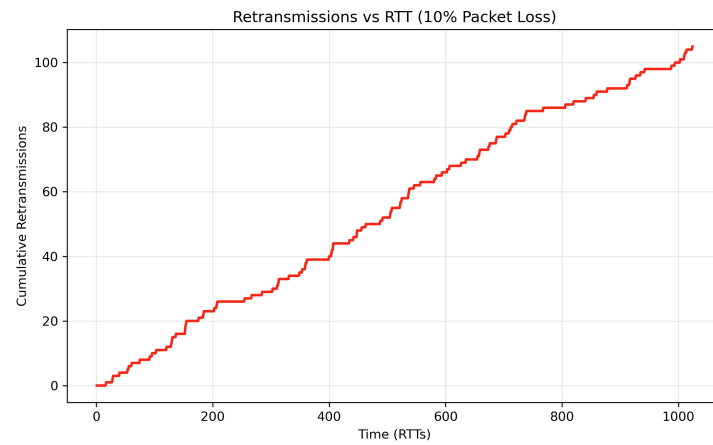


Figure 5: Retransmissions vs RTT with 10% packet loss

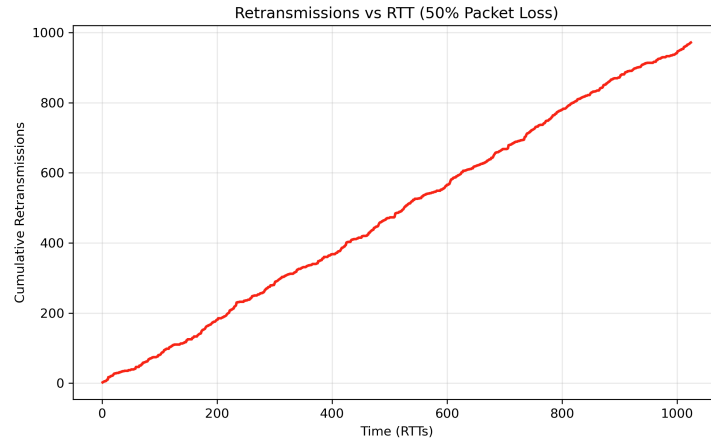


Figure 6: Retransmissions vs RTT with 50% packet loss

The retransmission graphs show cumulative retransmissions over time. At 1% loss, retransmissions increase slowly. At 10% loss, retransmissions increase more steadily. At 50% loss, retransmissions increase quickly because many packets or ACKs are lost.

6 Challenges and Lessons Learned

One challenge was handling cumulative ACKs correctly. The server needed to ACK only the highest contiguous byte received in order. Another challenge I faced was handling out-of-order packets. Specifically, this resulted in long wait times when testing. Buffering out-of-order data fixed this problem!

Another challenge was connecting congestion control to the sliding window. The client needed to track unacknowledged packets and only send up to `cwnd` packets at a time. This made `cwnd` actually control the sending rate.

Overall, I learned from this assignment on how to build TCP reliability on top of UDP using sequence numbers, ACKs, checksums, retransmissions, and congestion control. I worked on this assignment step-by-step, testing the client slide window, adding timeout retransmission and loss simulation before implementing congestion window functionality for ACKs. The step-by-step approach helped my implementation drastically.